

AUDITABLE AI FOR SUPPLY CHAIN FINANCE

One-page research brief for doctoral applications

Research question

How can a supply-chain finance decision connect temporal graph learning, an explicit policy boundary, role-governed business actions, and tamper-evident evidence without overstating what synthetic experiments prove?

Method

- Generate deterministic synthetic observations for 500 enterprises over 24 months and construct directed, time-valid relationships with 12-month graph windows.
- Compare XGBoost with a minimal GCN-BiLSTM temporal graph neural network. Export the promoted TGNN to ONNX and verify runtime parity.
- Route inference through a fixed policy layer, persist model/data/input identities, and atomically append assessment, decision, and workflow evidence to a PostgreSQL hash chain.
- Exercise five independent roles from application submission through audit, followed by a separate controlled disbursement, repayment, and closure simulation.
- Record controlled actual-outcome feedback for a closed facility, preserve immutable lineage, and fit a Platt calibration candidate for the business baseline.

Verified implementation

- Python 3.12, PyTorch research training, XGBoost comparison, ONNX Runtime inference, FastAPI, SQLAlchemy, PostgreSQL 17, Alembic, and bilingual browser acceptance.
- Reference dataset SHA-256: f784faa8bdef23625888d64de75c2f29a80c0a51e266e0822652569507648353.
- Promoted ONNX SHA-256: 158d273db310c3f1abf4be7cb06aee78568e475ebb7564ebbeaa16d3feeb0e5; parity maximum absolute error 2.38e-7.
- Tests cover temporal leakage, graph direction and normalization, artifact verification, authorization, concurrency, rollback, stale versions, duplicate invoices, ledger integrity, and five-role browser handoffs.
- Outcome evidence is hashed in the browser; calibration records sample composition, metrics, artifact integrity, and gated automatic activation or fallback.
- Optional single-organization Fabric 2.5.16 anchors hashes. Circom/Groth16 invoice $\leq$ limit evidence is wired into trade confirmation; Python checks structure, not cryptographic validity. Fabric does not verify proofs. This is not production ZKP or multi-organization consensus.

Synthetic evidence

- 2026_EXPLORATORY_SENSITIVITY; five deterministic seeds. Values are mean $\pm$ sample SD across $n=5$ seeds and are descriptive only.
- TGNN ROC-AUC 0.5469 ± 0.0855 (mean $\pm$ sample SD); PR-AUC mean is 0.1211. The mean is near chance and seed variability is high.
- XGBoost ROC-AUC 0.5889 ± 0.0202 (mean $\pm$ sample SD); PR-AUC mean is 0.1467. It is slightly steadier, not strong.
- At threshold 0.50, aggregate XGBoost positive-class recall 0.022 (FN 716, TP 16). Calibration deviation remains visible.
- Thresholds were fixed before reporting. No threshold is selected as optimal and no significance claim is made.

Limitations

- This implementation does not reproduce the original thesis: original data and code are unavailable, the TGNN is smaller, and the five-seed package is exploratory.
- Synthetic observations do not establish real-enterprise validity, causal benefit, fairness, robustness, production security, or economic value.
- The default PostgreSQL ledger is not blockchain consensus. The financing lifecycle does not execute real bank transfers and excludes interest, fees, FX, accounting, settlement, and reconciliation.
- Controlled/simulated outcome feedback does not retrain the TGNN, does not trigger on drift, and does not prove real-enterprise effects. Gated activation exists for the business baseline only; calibration candidates use chronological 70/30 holdout evaluation, and the retained corrected dataset is rejected before promotion because it lacks the required partition/class support.

Next research step

Run a preregistered, multi-institution study with governed real-enterprise outcomes, stronger temporal baselines, ablations, held-out calibration, drift monitoring, and decision-cost evaluation. Before any real deployment, require independent validation and human approval beyond the current demonstration gate.

The contribution is a traceable experimental and software boundary: every claim points to an artifact, test, or declared non-claim.